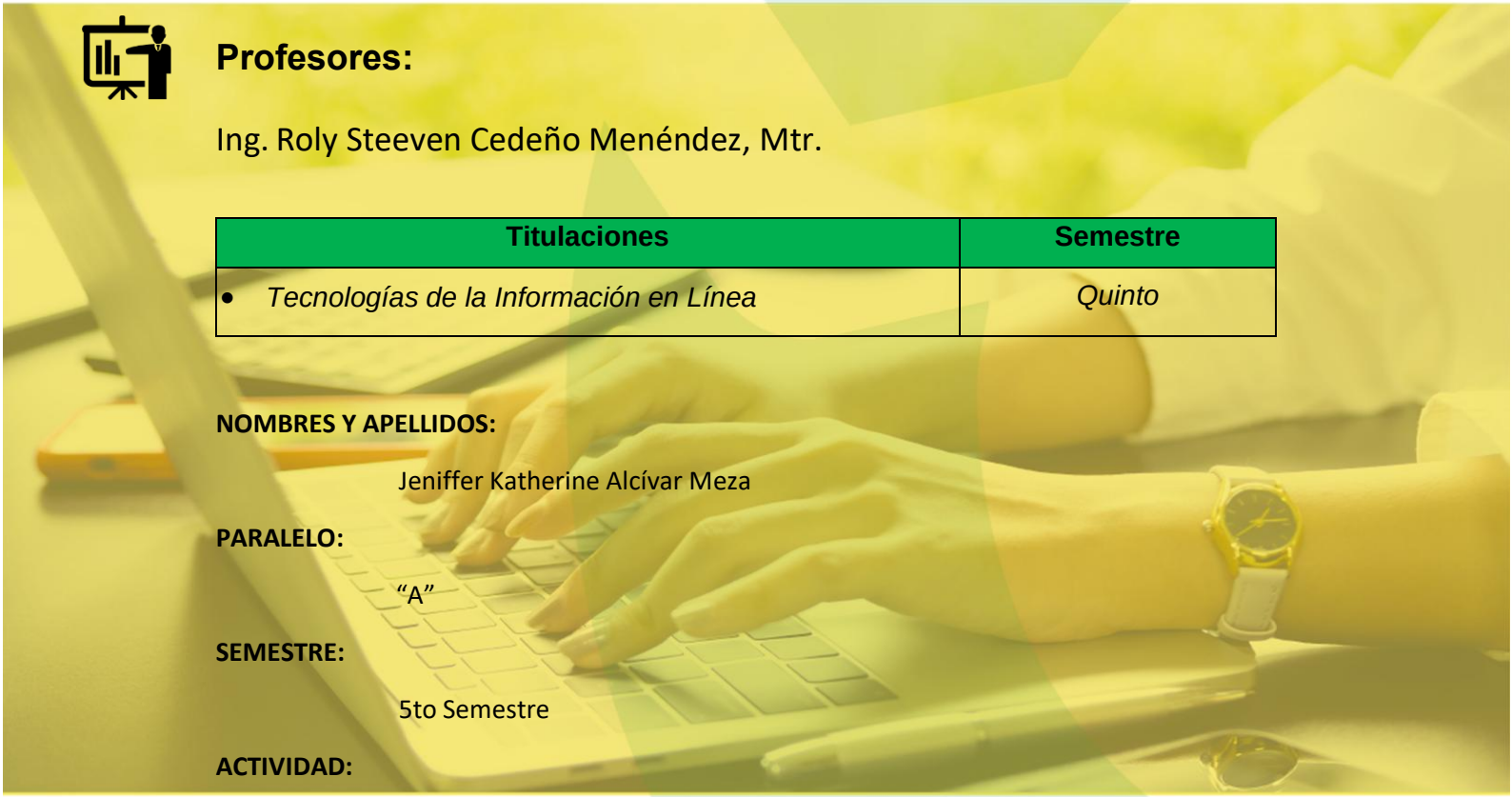
TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS:

Jeniffer Katherine Alcívar Meza

PARALELO:

"A"

SEMESTRE:

5to Semestre

ACTIVIDAD:

Sistema de Control de Rescate y Adopción Animal

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad 1

Actividad # 1

Enlace del repositorio: https://github.com/jenifferA-KM/Sistema-Rescate-adopci-n.git

Actores

- Ciudadano
- Voluntario
- Administrador

Requerimientos Funcionales

1. El sistema debe permitir registrar un nuevo caso con fecha, ubicación, descripción y datos del animal.
2. El sistema debe permitir clasificar al animal como perro, gato o fauna silvestre.
3. Debe permitir ingresar nombre o identificación, edad aproximada, condición física y fotografías, cuando estén disponibles.
4. El administrador debe poder asignar un voluntario o veterinario a cada caso.
5. El sistema debe permitir actualizar el estado del proceso: Rescatado, En tratamiento, En recuperación, Disponible para adopción o Adoptado.
6. El veterinario o personal autorizado debe poder registrar los tratamientos y cuidados proporcionados al animal.
7. Los usuarios autorizados deben poder buscar y consultar casos según su estado, especie o responsable.
8. El administrador debe poder consultar estadísticas de animales rescatados, en tratamiento y adoptados.
9. El administrador debe poder registrar, modificar o desactivar cuentas y asignar permisos según el rol.
10. El sistema debe guardar los cambios de estado, las asignaciones y las actuaciones realizadas en cada caso.

Requerimientos no funcionales

1. **Escalabilidad:** El sistema debe permitir aumentar la cantidad de casos, animales y usuarios sin que su rendimiento se deteriore significativamente.
2. **Mantenibilidad**: El sistema debe estar organizado y documentado para facilitar la corrección de errores, las actualizaciones y la incorporación de nuevas funciones.
3. **Trazabilidad de versiones:** El proyecto debe mantener un historial de cambios del código fuente y sus versiones mediante una herramienta como Git.
4. **Seguridad:** El sistema debe proteger la información mediante autenticación y permisos de acceso según el rol de cada usuario.
5. **Rendimiento:** Las consultas y operaciones habituales deben ejecutarse en un tiempo de respuesta adecuado, incluso cuando aumente la cantidad de registros.

6. **Disponibilidad:** El sistema debe estar disponible para los usuarios autorizados durante los horarios de operación del refugio, con interrupciones mínimas.
7. **Usabilidad:** La interfaz debe ser intuitiva, clara y fácil de utilizar para ciudadanos, voluntarios, veterinarios y administradores.
8. **Integridad de la información:** El sistema debe validar los datos ingresados y evitar registros incompletos o inconsistentes cuando sean obligatorios.
9. **Compatibilidad:** La aplicación web debe funcionar correctamente en navegadores modernos y adaptarse a computadoras, tabletas y teléfonos móviles.
10. **Respaldo y recuperación:** El sistema debe realizar copias de seguridad periódicas y permitir recuperar la información en caso de pérdida de datos.

Diagrama casos de uso

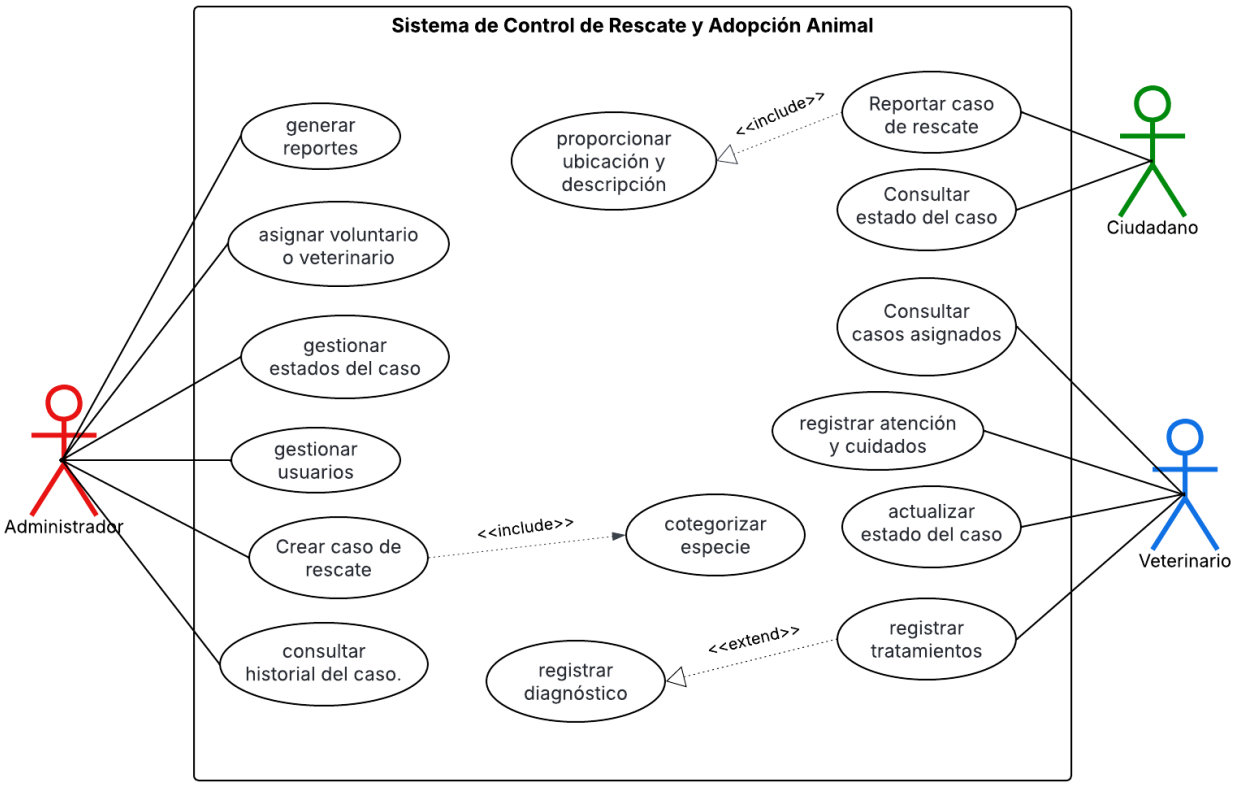
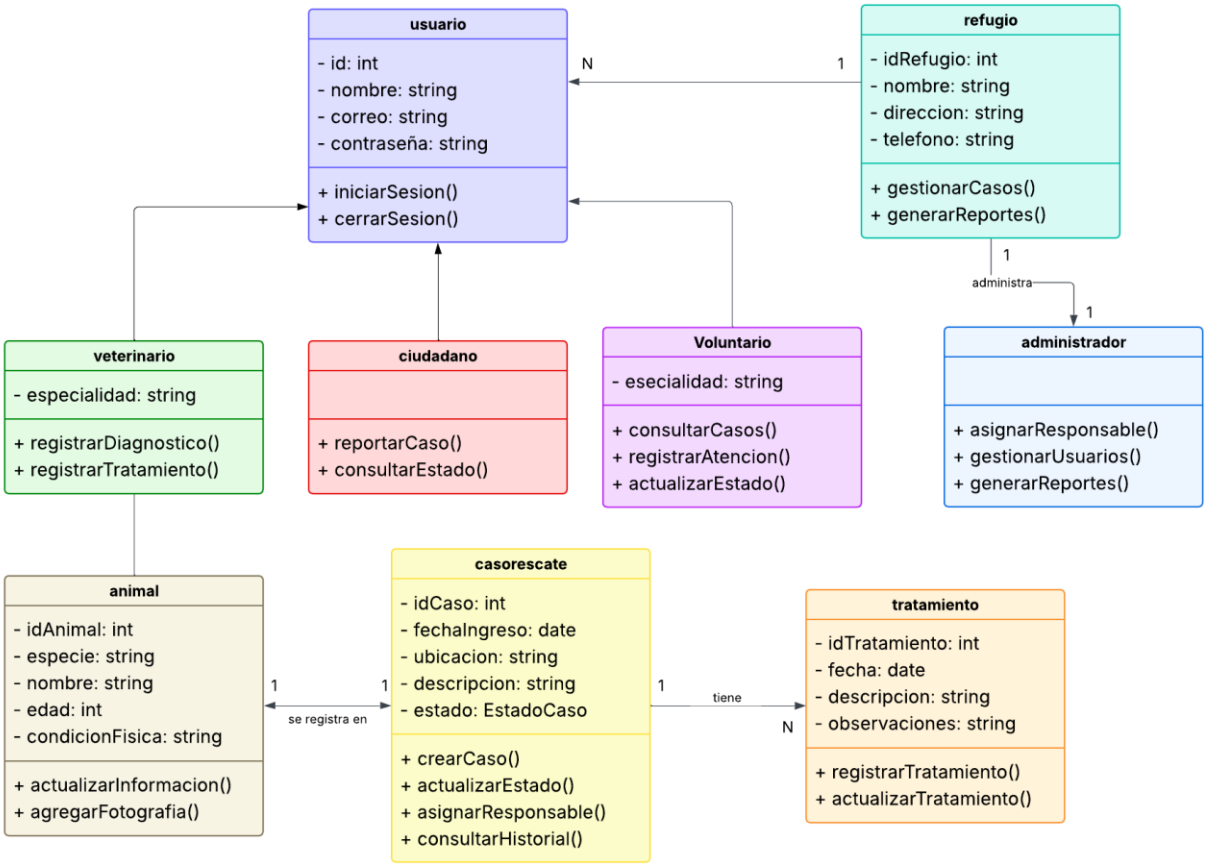


Diagrama UML

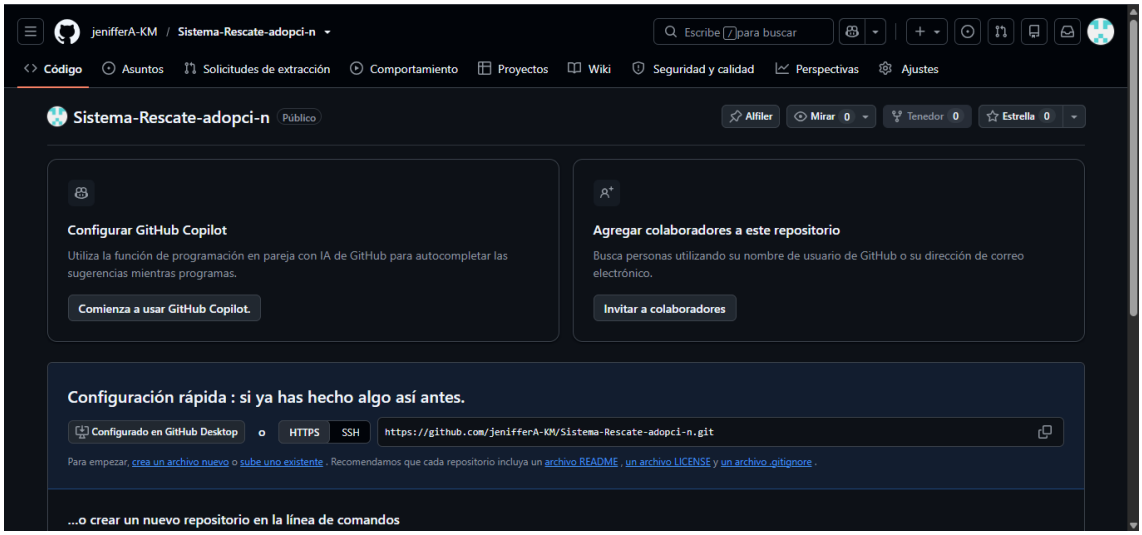


Justificación de los tres patrones escogidos

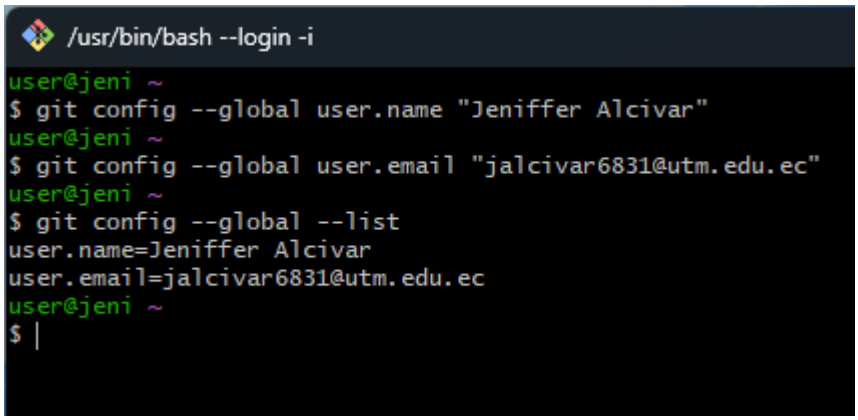
1. **Singleton:** Se escogió este patrón para mantener una única instancia del motor central de gestión de casos y coordinar el acceso a los datos compartidos. Esto beneficia al sistema al evitar la creación innecesaria de múltiples instancias y centralizar determinadas operaciones.
2. **Observer:** Se eligió este patrón para enviar notificaciones automáticas a los voluntarios y veterinarios cuando se registre un nuevo caso de rescate o se produzcan cambios relevantes. Su beneficio es agilizar la comunicación, mejorar la coordinación del equipo y permitir una respuesta más rápida ante las emergencias.
3. **Factory Method:** Se seleccionó este patrón para crear expedientes según la especie del animal rescatado, como perros, gatos o fauna silvestre. Esto facilita adaptar la información y los procedimientos a cada tipo de animal, además de permitir incorporar nuevas especies sin modificar toda la lógica del sistema.

Práctica GitHub

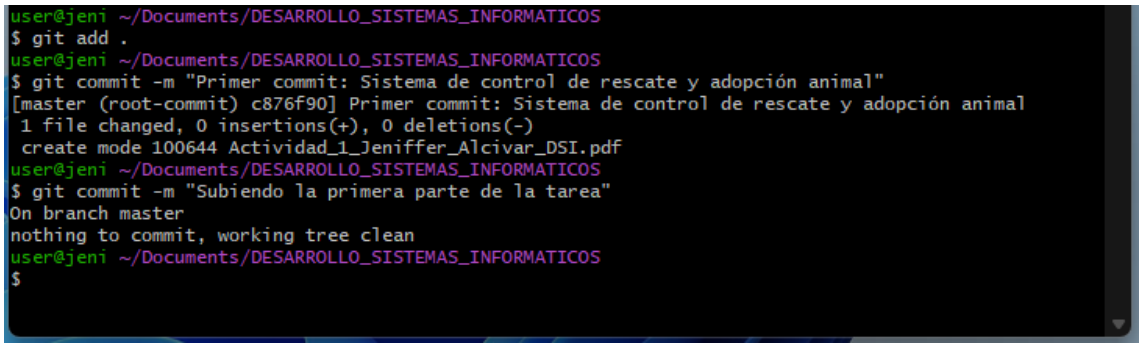
Crear un repositorio en GitHub



Abrimos el git para configurar la identidad localmente.



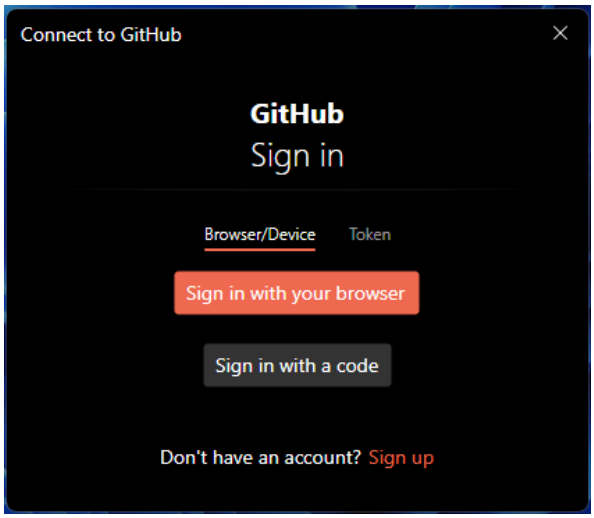
Crear una carpeta en documentos y colocar el PDF. Damos clic derecho sobre dicha carpeta y abrimos la consola de git bash. Iniciamos y realizamos el primer commit.



Después vinculamos la carpeta local al GitHub

```
user@jeni ~/Documents/DESARROLLO_SISTEMAS_INFORMATICOS
$ git remote add origin https://github.com/jenifferA-KM/Sistema-Rescate-adopci-n.git
user@jeni ~/Documents/DESARROLLO_SISTEMAS_INFORMATICOS
$ git branch -M main
user@jeni ~/Documents/DESARROLLO_SISTEMAS_INFORMATICOS
$ git push -u origin main
```

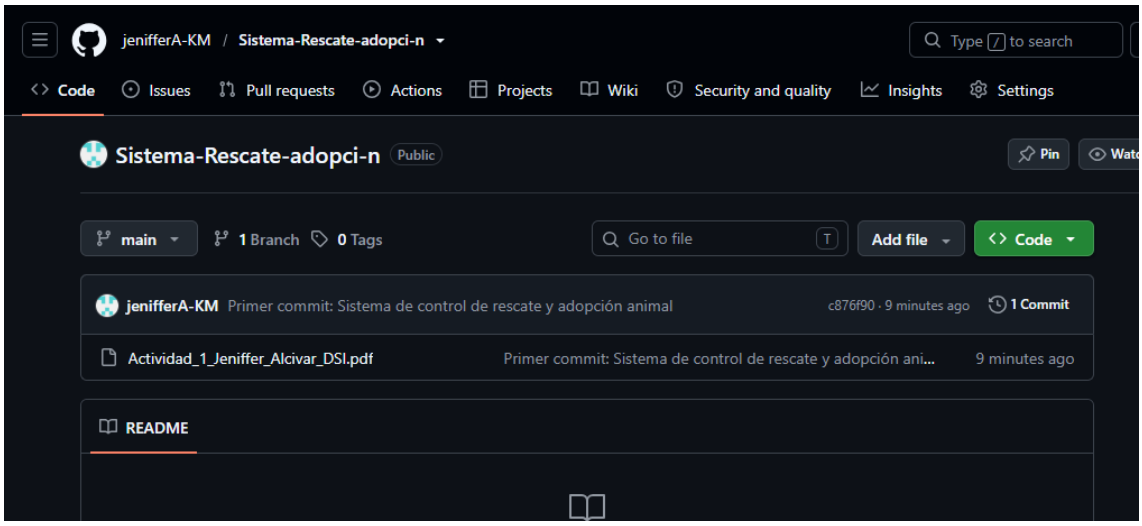
Saldrá una pequeña ventana que nos pide que ingresemos a nuestra cuenta de GitHub.



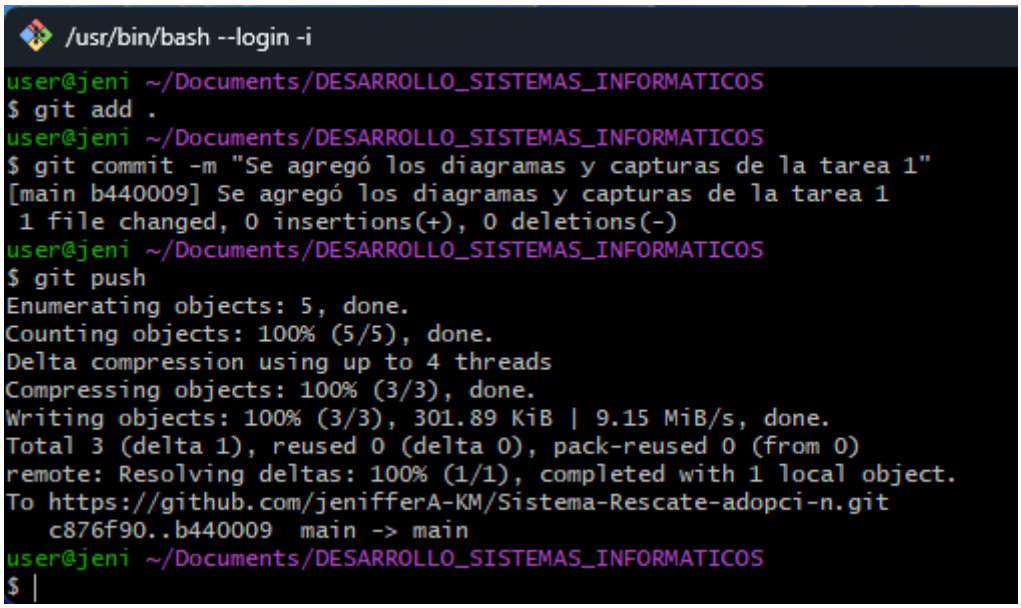
Y autorizamos con el código único que llegará al correo



Actualizamos la página del GitHub y aparecerá el PDF junto al commit que se realizó



Realizamos cambios en el pdf y reemplazamos, luego hacemos otro commit



Recargamos nuevamente la página y veremos los dos commit que hizo.

